

PROMPT IA 01 - MONOREPO

BACKEND E-COMMERCE

Construire le socle monorepo Go et les standards de developpement

Usage	A coller tel quel dans une IA de developpement / coding agent.
Projet	Plateforme e-commerce multi-site OSS-first, Go + Kubernetes + GitOps.
Mode	Execution autonome, incrémentale, testee, sans sur-ingenierie.
Priorite	Code propre, factorise, maintenable, securise et observable.

IMPORTANT: ne pas remplacer les choix techniques valides par des alternatives sans blocage objectif.

1. Mission de l'agent IA

Tu es un lead backend/platform engineer. Ta mission est de creer un monorepo Go de production pour une plateforme e-commerce microservices. Tu dois produire du code executable, teste, documente et facile a maintenir. Tu travailles de maniere incrementale et tu n'ajoutes aucune dependance ou abstraction sans justification technique concrete.

Regle de travail

Ne fais pas un prototype jetable. Chaque fichier cree doit pouvoir servir de base aux services suivants. Ne genere pas les 17 microservices complets dans ce prompt : construis le socle commun, le tooling, la structure et les conventions qui permettent de les ajouter proprement.

2. Contexte d'architecture a respecter

- Backend standardise en Go.
- REST/JSON pour les API externes, gRPC pour les appels synchrones service-to-service.
- Kafka via franz-go pour les evenements metier durables ; RabbitMQ pour les jobs/work queues.
- PostgreSQL via pgx + sqlc ; CloudNativePG en production ; 1 base logique par microservice.
- Redis pour cache/hot reads/rate limiting, jamais pour les sessions d'authentification.
- OpenTelemetry, slog JSON, Prometheus, Loki, Tempo.
- OpenAPI 3.1 pour REST ; Protobuf + Buf pour gRPC et evenements Kafka.
- Outbox Pattern, idempotence, retries bornes, DLQ, correlation_id et trace_id.
- Images OCI multi-stage/distroless, execution non-root et filesystem read-only.
- CI Tekton ; Harbor ; Trivy ; Syft/SBOM ; Cosign ; FluxCD ; Flagger.
- Git est la source de verite. Aucun secret applicatif longue duree dans Git ou Kubernetes.
- Architecture Clean/Hexagonale : domain -> application -> infrastructure -> transport.

3. Objectif fonctionnel du monorepo

Le repository doit permettre de creer rapidement un nouveau microservice Go sans copier des centaines de lignes ni dupliquer la logique transversale. Les bibliotheques partagees doivent rester minimales et techniques uniquement.

```
ecommerce-backend/  
├── go.work  
├── Makefile  
├── README.md  
├── .editorconfig  
├── .golangci.yml  
├── buf.work.yaml  
├── services/  
│   ├── product/  
│   ├── catalog/  
│   ├── pricing/  
│   ├── inventory/  
│   └── ...  
└── libs/
```

Prompt de developpement IA - architecture e-commerce

```

├── observability/
├── messaging/
├── problem/
├── testkit/
├── contracts/
│   ├── proto/
│   ├── events/
│   └── openapi/
├── build/
│   ├── docker/
│   └── tekton/
├── deploy/
│   ├── base/
│   └── overlays/
├── tools/
│   └── tools.go
├── docs/
│   ├── adr/
│   └── conventions/

```

4. Contraintes de conception

- Aucune logique métier dans libs/. Une lib partagée ne doit contenir que du code transversal stable.
- REST et gRPC doivent appeler les mêmes use cases. Aucun doublon de logique métier.
- Chaque service possède son go.mod et ses migrations.
- go.work relie les modules locaux pour le développement.
- Les contrats Protobuf doivent être versionnés et vérifiés avec Buf breaking checks.
- Les erreurs applicatives utilisent des codes stables, mappés proprement vers HTTP/gRPC.
- Tous les appels externes ont context.Context, deadline et timeout.
- Pas de réflexion, framework magique ou DI container lourd. Préférer des constructeurs explicites.
- Pas de global mutable state.
- Aucune fonction ne doit faire à la fois validation, transaction DB, appel réseau et publication d'événement.

5. Livrables de code obligatoires

1. Initialiser le repository et go.work.
2. Créer une convention de module/service reproductible.
3. Créer les libs techniques minimales : observability, error/problem mapping, messaging envelope, testkit.
4. Configurer golanci-lint, gofmt, go vet, tests, race detector et couverture.
5. Configurer Buf pour lint/generation/breaking checks.
6. Fournir un Makefile racine factorisé qui boucle sur les modules sans dupliquer les commandes.
7. Fournir un Dockerfile de référence multi-stage et distroless, non-root.
8. Fournir des templates Tekton de CI réutilisables.
9. Fournir les manifests GitOps de base Helm/Kustomize sans secrets.

10. Fournir une documentation courte de creation d'un nouveau service.

6. Makefile et automatisation

Le Makefile racine doit decouvrir les modules automatiquement et appliquer les commandes via des fonctions/boucles, sans copier la meme cible pour chaque service.

```
make fmt
make lint
make test
make test-race
make test-integration
make generate
make buf-lint
make buf-breaking
make build
make docker
make verify
```

La cible verify doit executer tous les controles bloquants locaux dans un ordre court-circuitable et retourner un code de sortie non nul au premier echec critique.

7. Bibliotheques partagees autorisees

Lib	Contenu autorise	Contenu interdit
observability	OTel init, trace/log correlation, metrics helpers	business spans specifiques a un domaine
messaging	envelope Kafka, headers, idempotency primitives	evenements Product/Order eux-memes
problem	codes techniques, mapping HTTP/gRPC	regles metier
testkit	containers/helpers/factories techniques	fixtures metier globales

8. Contrats et versioning

- OpenAPI 3.1 : lint et detection de breaking change en CI.
- gRPC : package service.v1 ; Buf lint + breaking.
- Kafka : Protobuf + enveloppe standard event_id, event_type, aggregate_id, aggregate_version, occurred_at, producer, correlation_id, causation_id, trace_id.
- Les IDs nouveaux sont UUIDv7.
- Temps en UTC, RFC3339 aux interfaces.

Prompt de developpement IA - architecture e-commerce

- Argent sans float : amount_minor entier + currency ISO 4217.

9. Standards de test

Type	Execution	Attendu
Unitaires	chaque PR	rapides, sans reseau, table-driven
Integration	chaque PR	testcontainers PostgreSQL/Kafka si requis
Contracts	chaque PR	REST/gRPC/Kafka compatibles
Race	chaque PR backend	go test -race
Security	chaque build	Trivy/SBOM/licences

Quality gate

Aucun merge si fmt, lint, tests, race, contracts, build ou security gate echoue. Les warnings non critiques sont autorises uniquement s'ils sont explicitement documentes.

10. Securite by default

- runAsNonRoot=true, allowPrivilegeEscalation=false, capabilities drop ALL, seccomp RuntimeDefault.
- readOnlyRootFilesystem=true ; tmpfs/emptyDir uniquement si necessaire.
- Aucun secret dans les fichiers de config ou exemples commitables.
- OpenBao est la source de verite des secrets runtime.
- Les logs ne contiennent ni JWT complet, ni PAN/CVV, ni secrets, ni PII inutile.
- Les services n'exposent que /health/live, /health/ready et /health/startup comme endpoints techniques.

11. Observabilite minimale integree

Le socle doit fournir une initialisation uniforme des logs, traces et metriques, sans masquer le contexte du service.

```
log fields minimum:
- service
```

```
- version
- environment
- trace_id
- correlation_id
- request_id

metrics minimum:
- http_server_requests_total
- http_server_duration_seconds
- grpc_server_requests_total
- grpc_server_duration_seconds
- dependency_errors_total
```

12. Conventions de base de donnees

- pgxpool borne ; aucune connexion illimitée.
- sqlc obligatoire pour les requetes applicatives.
- Aucun appel reseau dans une transaction PostgreSQL.
- Transactions courtes : ecriture metier + outbox puis COMMIT.
- Migrations backward-compatible selon expand -> migrate -> contract.
- Aucune FK cross-service.
- statement_timeout et idle_in_transaction_session_timeout configurables.

13. Messaging

Le monorepo doit fournir les primitives, pas les evenements metier globaux.

- Kafka = evenements durables et integration asynchrone.
- RabbitMQ = jobs/work queues/retries differes/priorites.
- Outbox transactionnelle pour les evenements produits depuis PostgreSQL.
- Consumer idempotent par event_id.
- DLQ avec raison d'echec et metadonnees de correlation.
- Partition key Kafka = aggregate_id.

14. CI/CD et artefacts

```
PR
↓
gofmt / go vet / golangci-lint
↓
unit + integration + race + contracts
↓
build OCI
↓
Trivy + Syft/SBOM + license policy
↓
Cosign
↓
Harbor @sha256
↓
FluxCD
↓
Flagger si Tier critique
```

Le prompt ne demande pas de deployer un cluster reel. Produis les fichiers declaratifs et les scripts/targets necessaires pour que la pipeline puisse les utiliser.

15. Documentation attendue

- README racine : prerequis, bootstrap, commandes principales.
- docs/conventions/backend.md : architecture, naming, erreurs, timeouts, IDs, argent, temps.
- docs/conventions/testing.md : pyramide de tests et quality gates.
- docs/conventions/messaging.md : Kafka/RabbitMQ/Outbox/idempotence.
- ADR initial expliquant le choix monorepo + go.work et la politique libs communes minimales.

16. Plan d'execution impose a l'IA

1. Inspecter le repository existant avant toute modification.
2. Proposer un plan de 5 a 10 etapes maximum.
3. Executer une seule etape a la fois, en gardant le repository compilable.
4. Apres chaque etape : lancer les tests pertinents et corriger immediatement.
5. Factoriser avant d'ajouter un copier/coller identique dans un deuxieme service.
6. Ne jamais supprimer un fichier existant sans verifier ses usages.
7. A la fin : executer make verify et fournir le resultat exact.

17. Definition of Done

Controle	DoD
Build	Tous les modules Go compilent.
Lint	gofmt, go vet et golangci-lint passent.
Tests	Unit, integration et race passent.
Contracts	Buf/OpenAPI checks passent.
Docker	Image non-root buildable et minimale.
CI	Pipeline declarative exploitable.
Docs	Nouveau service reproductible sans connaissance implicite.
Maintainability	Pas de duplication evidente ni de framework inutile.

18. Prompt final a executer

Instruction a l'agent IA

Commence maintenant l'implementation du monorepo. Ne me redemande pas les choix d'architecture deja precises ci-dessus. Si un detail manque, choisis l'option la plus simple, portable et standard, documente-la dans un ADR seulement si elle est structurante. Termine par un recapitulatif des fichiers modifies, des tests executes, des risques restants et de la prochaine etape conseillee.